UNIVERSITY OF PENNSYLVANIA

ESE 650: LEARNING IN ROBOTICS

MIDTERM EXAM

03/31 MON 10.15 AM – 11.45 AM ET

DURATION: 75 MINUTES + 15 MINUTES TO SCAN

MAXIMUM POINTS: 72

Read the following instructions before you begin.

- **WRITE YOUR NAME AND PENN EMAIL AT THE TOP ON THIS PAGE.**
- This is a closed book exam. You are allowed to use one A4/Letter paper cheat sheet (front and back).
- You may not use laptops/phones/tablets or the Internet to look up a textbook/notes/or anything else during this exam. You may not discuss with your peers.
- You will have 75 minutes from the time you start the exam. After that you will get 15 minutes to upload your solutions to Gradescope.
- You should use a pen and paper (do not use a pencil) to write your solutions and click pictures and upload your solutions as a PDF (Dropbox or Apple Notes apps on your phone are great for scanning all pages as a single PDF).
- You should bring a phone/tablet to take pictures of your exam. You will then click pictures of each page and upload these pages to Gradescope from the phone, or after transferring this to your laptop/tablet if you wish.
- **YOU SHOULD SCAN EVERY SINGLE PAGE OF THE EXAM, INCLUDING THIS FIRST PAGE AND EVEN THE QUESTIONS THAT YOU DID NOT ANSWER.**
- Each question mentions how short/long we would like your responses to be. DO NOT write excessively verbose answers. None of the questions require long derivations. If you are stuck, explain your answers and what you are trying to do clearly. We will give partial credit if you are on the right track.
- **This exam has 5 problems.** The questions are NOT arranged in order of difficulty. Try to attempt every question.

Problem 1 (18 points).

(1) **(2 points)** Answer True or False with 1-2 sentence explanation. The forward algorithm in an HMM computes $\alpha_k(x) = \mathrm{P}(Y_1, \ldots, Y_k, X_k = x)$. The values of the forward variables are normalized: $\sum_x \alpha_k(x) = 1$.

> **Answer:** No. It is a joint probability of the observation sequence.

(2) **(4 points)** Describe in 4-5 sentences how you will use dynamic programming to compute the top 5 most likely trajectories $(x_1^{(i)}, \ldots, x_k^{(i)})$ for $i = 1, \ldots, 5$ ordered by their decoding probability

$$\mathrm{P}(X_1 = x_1^{(i)}, \ldots, X_k = x_k^{(i)}, Y_1, \ldots, Y_k).$$

> **Answer:** Should describe how they will maintain 5 different parents for each node in Viterbi's algorithm. Should calculate the 5 probabilities of arriving at the current node in the Trellis graph. It is okay if students do not mention the equations so long as they are doing it correctly.
>
> $$\delta_{k+1}^{(i)}(x) = \max_{x',j}^{(i)} \delta_k^{(j)}(x') T_{x'x} M_{x,y_{k+1}}$$
>
> $$\text{parent}_{k+1}^{(i)}(x) = \operatorname*{argmax}_{x',j}^{(i)} \delta_k^{(j)}(x') T_{x'x}.$$
>
> where the superscripts $^{(i)}$ denote the ith most likely parent/or its probability. The notation $\max_{x',j}^{(i)}$ denotes that we first calculate all the terms for different x', j and then take the ith largest one.
>
> No points if people simply say "dynamic programming".

(3) Let us invent a new algorithm for filtering. It works as follows.

(a) **(2 points)** First compute

$$\mathrm{P}(X_k = x \mid Y_1, \ldots, Y_k)$$

What algorithm will you use to do this. Answer in 1-2 sentences.

> **Answer:** This is standard filtering. Use the Bayes filter. Note, not the forward algorithm.

(b) **(2 points)** Draw one sample from the sequence of future states.

$$(\hat{x}_{k+1}, \hat{x}_{k+2}, \ldots, \hat{x}_{k+T}) \sim \mathrm{P}(X_{k+1}, \ldots, X_{k+T} \mid Y_1, \ldots, Y_k).$$

What algorithm will you use to do this. Answer in 1-2 sentences.

Answer: This is standard prediction. First run Bayes filter. Then the prediction algorithm for T time-steps with $\mathrm{P}(X_k = x \mid Y_1, \ldots, Y_k)$ as the initial condition.

(c) **(2 points)** Calculate the most likely future sequence of observations for these sampled states

$$(\hat{Y}_{k+1}, \hat{Y}_{k+2}, \ldots, \hat{Y}_{k+T}) = \operatorname{argmax} \mathrm{P}(Y_{k+1}, \ldots, Y_{k+T} \mid X_{k+1} = \hat{x}_{k+1}, \ldots, X_{k+T} = \hat{x}_{k+T}, Y_1, \ldots, Y_k).$$

How will you do this? Answer in 1-2 sentences.

Answer: Each observation is conditionally independent of future and past given the state. So we can just sample from the HMM's emission matrix to calculate the most likely observation at each step.

(d) **(2 points)** Answer in 1-2 sentences how you will calculate the expression

$$\mathrm{P}(X_k = x \mid Y_1, \ldots, Y_k, \hat{Y}_{k+1}, \hat{Y}_{k+2}, \ldots, \hat{Y}_{k+T}).$$

Answer: This is standard smoothing. So run the forward-backward algorithm for $k + T$ steps, calculate the product of $\alpha_k(x)\beta_k(x)$ at time k, normalize it to get the required estimate.

(e) **(4 points)** Answer True or False with 1-2 sentence explanation which one of steps (a) and step (d) will give a more precise estimate of the state at time k.

$$\mathrm{P}(X_k = x \mid Y_1, \ldots, Y_k) \text{ or } \mathrm{P}(X_k = x \mid Y_1, \ldots, Y_k, \hat{Y}_{k+1}, \hat{Y}_{k+2}, \ldots, \hat{Y}_{k+T}).$$

Write down the expression for how you will modify the right-hand side expression to make it equal to the left-hand side one.

Answer: The two are not equal, the right-hand side is conditioned on more observations. Effectively we are hallucinating more observations. Therefore, the filtering estimate is still the more precise estimate of the state at time k. The two would only be equal if we marginalize over all hallucinated futures,

$$\mathrm{P}(X_k = x \mid Y_1, \ldots, Y_k) = \mathop{\mathrm{E}}_{\hat{Y}_{k+1}, \ldots, \hat{Y}_{k+T}} \left[\mathrm{P}(X_k = x \mid Y_1, \ldots, Y_k, \hat{Y}_{k+1}, \hat{Y}_{k+2}, \ldots, \hat{Y}_{k+T}) \right].$$

It does not matter how the hallucinated futures are sampled.

Problem 2 (15 points).

(1) **(4 points)** Consider a fixed one-dimensional state $x \in \mathbb{R}$. We obtain observations of this state $y_i = x + \nu_i$ for n time-steps where the observation at the ith time is $y_i \in \mathbb{R}$ corrupted by standard Gaussian noise $\nu_i \sim N(0, 1)$. Calculate the analytical expression of the probability distribution of x after n time-steps. The initial distribution of $x \sim N(0, \sigma^2)$ with standard deviation $\sigma > 0$. You might find the equations for a Kalman filter useful

$$\begin{aligned}
\mu_{k+1|k} &= A\mu_{k|k} + Bu_k \\
\Sigma_{k+1|k} &= A\Sigma_{k|k}A^\top + R \\
K_{k+1} &= \Sigma_{k|k}C^\top \left(C\Sigma_{k|k}C^\top + Q\right)^{-1} \\
\mu_{k+1|k+1} &= \mu_{k+1|k} + K_{k+1}(y_{k+1} - C\mu_{k+1|k}) \\
\Sigma_{k+1|k+1} &= \left(\Sigma_{k+1|k}^{-1} + C^\top Q^{-1} C\right)^{-1}.
\end{aligned}$$

Answer: Let $x \sim N(\mu_n, \sigma_n^2)$ and let $y = [y_1, \ldots, y_n] \in \mathbb{R}^n$ be the observations arranged in one vector. This is like having one n-dimensional observation with $C = [1, \ldots, 1]^\top \in \mathbb{R}^n$, the covariance $Q = I$. Only the last two equations of the Kalman filter are necessary.

$$\begin{aligned}
k &= C^\top \left(CC^\top + \sigma^2 I\right)^{-1} \\
\mu_n &= ky \\
\frac{1}{\sigma_n^2} &= \frac{1}{\sigma^2} + n.
\end{aligned}$$

(2) **(2 points)** Answer True or False with 1-2 sentence explanation. An unscented Kalman filter can be used when the dynamics is linear, observations are linear in the state and both dynamics and observations are corrupted by Gaussian noise.

Answer: True. A UKF is a generalization of the KF.

(3) **(2 points)** Answer with a 1-2 sentence explanation. For what kinds of problems can we run a particle filter without the resampling step.

Answer: For low-dimensional problems where the dynamics and observation noise are such that particles do not "accumulate" in one specific part of the state space. This happens when the dynamics mixes sufficiently and can go to many other states from each state, and the observation likelihood is rather diffusion, i.e., many different states can explain the observation.

(4) **(3 points)** Answer with a 2-3 sentence explanation. Explain the key ideas behind an extended Kalman filter. You can explain using the Kalman filtering equations given above in part (1). Explain when you would prefer to use an extended KF instead of an UKF, even if the system is such that both are applicable.

Answer: An EKF linearizes the dynamics and observations at each timestep and uses the KF equations to update the mean and covariance of the state estimate. An EKF only works for Gaussian noise, although it handles nonlinear dynamics and observations.

(5) **(2 points)** Answer True of False with a 1-2 sentence explanation: A standard Kalman filter is the optimal linear filter even for nonlinear systems with Gaussian noise.

Answer: True. A standard KF optimizes the gain to minimize the mean squared error. So it is the best linear filter even if the system is nonlinear.

(6) **(2 points)** To even attempt the procedure suggested in the previous question, we need to choose a fixed value for the A, B and C matrices to run the standard KF for the nonlinear system. We need a mechanism to score a particular choice of matrices $A^{(1)}, B^{(1)}, C^{(1)}$ against some other choice $A^{(2)}, B^{(2)}, C^{(2)}$. Without access to the "ground-truth" state in typical filtering problems, how can we decide which of these two systems is better for running the KF?

Answer: In an optimal KF, the innovation is white noise, i.e., uncorrelated in time and fixed variance. We can compute the auto-correlation function of the innovation and pick the configuration which is closest to white noise.

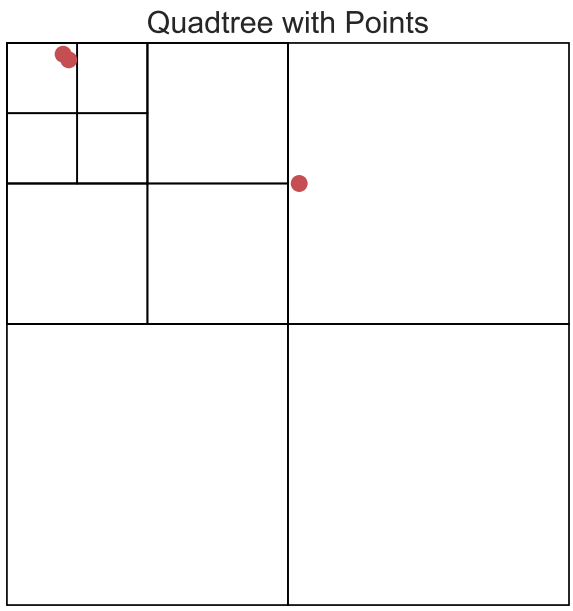
Problem 3 (15 points).

(1) **(4 points)** Consider a set of three 2-dimensional points

$$\{(0.52, 0.75), (0.11, 0.97), (0.1, 0.98)\}.$$

We would like to build a quadtree from these points. The domain of the quadtree is $[0, 1] \times [0, 1]$, i.e., all points will have both coordinates in $[0, 1]$. The finest cell in the quadtree is of size $[1/8, 1/8]$. Draw the quadtree that results from inserting these three points.

Answer:



(2) **(3 points)** Describe in 2-3 sentences how and why we use position encoding in a NeRF.

Answer: To model fine-scale texture and shapes in the scene, a NeRF needs to predict very different colors or opacity for nearby locations. This entails that the function being fitted is not smooth. Typically neural networks are inadequate for fitting such non-smooth functions. Position encoding is used to modify the

input representation of the location. For each coordinate (both locations and viewing directions), say, x we use a set of frequencies and convert it into a feature vector $[\cos(\omega_1 x), \sin(\omega_1 x), \cos(\omega_2 x), \sin(\omega_2 x), \dots]$.

Important to mention viewing direction.

(3) **(2 points)** Explain in 1-2 sentences what is a pure quaternion.

Answer: A point $x \in \mathbb{R}^3$ corresponds to a pure quaternion $(0, x)$. This is useful because we can transform the point by the rotation matrix represented by the quaternion using the expression $q \cdot (0, x) \cdot q^*$.

(4) **(2 points)** Answer True or False with 1-2 sentence explanation. The space of rotation matrices in $SO(3)$ is 3 dimensional.

Answer: True. For example, quaternions are 4 numbers but they are normalized.

(5) **(2 points)** Answer True or False with a 1-2 sentence explanation. The two unit quaternions $q = (u_0, u_1, u_2, u_3)$ and $q' = (-u_0, -u_1, -u_2, -u_3)$ parameterize the same 3D rotation.

Answer: True. Quaternions are a double cover of $SO(3)$. The negative quaternion q' corresponds to the same rotation because it has an axis pointing in the opposite direction as that of q, and an angle that is also the negative of the angle of q.

(6) **(2 points)** Consider a robot that lives in SE(3). At time zero, the body frame and the world frame coincide, in particular the coordinates of the robot in the world frame at time zero $x_0 = (0, 0, 0)$. Suppose it took two steps SE$(3) \ni g_1 = (R_1, t_1)$ and SE$(3) \ni g_2 = (R_2, t_2)$ where R, t are rotation and translation measured in the body frame. Calculate

$$g_2 \circ g_1,$$

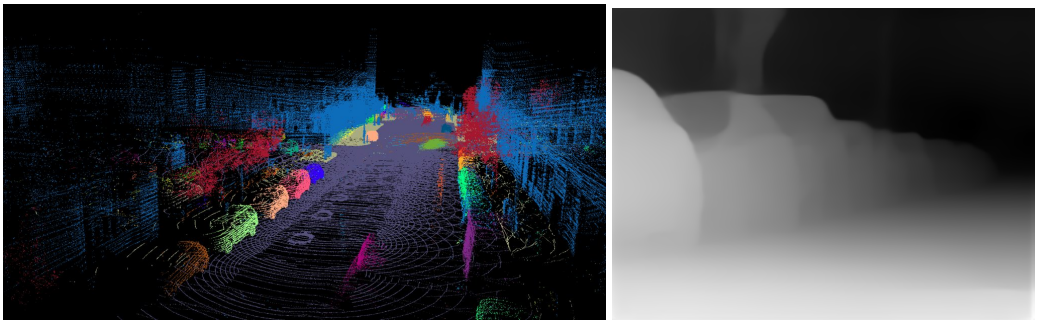
i.e., the transformation from the body frame to the world frame explicitly in terms of (R_k, t_k)s. Explain your answer.

Answer: After the first step, the transformation between the body frame and the world frame is $g_1 = (R_1, t_1)$. So a point x in body frame gets mapped to $R_1 x + t_1$ in the world frame. If we further compose a transformation g_2, the

point x is mapped to $R_2(R_1x + t_1) + t_2$, i.e.,

$$g_2 \circ g_1 = (R_2R_1, R_2t_1 + t_2).$$

Problem 4 (12 points).
A LiDAR returns a sparse set of points in the 3D world. By itself one LiDAR observation does not have sufficient structure to classify different categories of objects with good precision, or their locations precisely. This also creates problems while registering RGB camera images to LiDAR data etc.



Answer the following questions.

(a) **(4 points)** Assume that you have access to m observations from viewpoints $x_i \in \mathrm{SE}(3)$ for $i = 1, \ldots, m$. Each observation y_i has n points at distances d_i^j along different rays ω_i^j of a 3D LiDAR, we can denote this as a set $y_i = \{(d_i^j, \omega_i^j)\}_{j=1}^n$. Write the mathematical expression for the accumulated point-cloud, i.e., the point-cloud that combines all m observations.

(b) **(8 points)** The point-cloud you obtained in the first step looks like the one shown in the image on the left-hand side. It still has a few "holes". Your next goal is to combine these scans and output a higher resolution point-cloud (a set of points in 3D, not just images) that is better than simply combining the points like we did in the previous step. Your point-cloud should look like the one shown on the right above, i.e., much smoother 3D surfaces with marginal or no "holes".

Assume that our viewpoints are rich enough to give us observations of the scene from diverse viewpoints. You should provide sufficient details to convince the reader that your approach is correct.

Answer: (a) The point cloud is

$$P = \left\{ x_i + d_i^j \omega_i^j \,:\, i = 1, \ldots, m, \text{ and } j = 1, \ldots, n \right\}.$$

because the direction is ω_i^j and d_i^j is the distance along it. Take off points if people have made mistakes in this expression.

(b) We can build a NeRF (no need for colors, we only need the density). We should supervise the training of this NeRF using the points in the point cloud. Everything else remains the same. We still use position encoding. The loss changes, it is no longer MSE loss on the rendered images. It is instead MSE loss on the points in the point cloud. No need to write mathematical expressions. It is okay if people explain this in words. The important point is to notice that the NeRF will be forced to average the occupancy (because it is fitting a neural network). This allows the network to "in-fill" the holes in the point cloud. The next point that students should mention is that they should say how to calculate the new 3D point cloud. They should query the NeRF at a very dense set of points. Or say smoothing like, they will query the NeRF at any point to get the point cloud (like a signed distance function).

Problem 5 (12 points).

(1) **(2 points)** For a deterministic system n states and m controls that can be taken at each state, for a time-horizon of T time-steps, what is the computational complexity of running dynamic programming?

> **Answer:** It is $\mathcal{O}(Tmn)$ because we are computing a deterministic controller, not a feedback control.

(2) **(5 points)** Suppose we have a Markov Decision Process with n states denoted by x and m controls denoted by u, let us denote it abstractly as

$$x_{k+1} = f(x_k, u_k) + \epsilon_k$$

where ϵ_k is the noise at time-step k and $f(\cdot, \cdot)$ denotes the dynamics. Describe an algorithm to compute the best feedback control $u^*(\cdot)$ that optimizes the objective

$$J(x_0; u(\cdot)) = \lim_{T \to \infty} \mathop{\mathrm{E}}_{\epsilon_0, \epsilon_1, \ldots, \epsilon_{T-1}} \left[\sum_{t=0}^{T-1} \gamma^k q(x_k, u(x_k)) \right]$$

for some $\gamma < 1$ and initial state x_0.

> **Answer:** Value iteration or policy iteration. See lecture notes. Check for whether the initialization, expression for per-step iteration and termination criterion are correct.

(3) **(2 points)** Answer True of False with a 1-2 sentence explanation. For an infinite-horizon stochastic dynamic programming problem with finite states and finite controls, even if we only have a terminal reward, i.e., a reward of 1 if the robot reaches a goal state, the cost to go from every state is non-zero.

> **Answer:** True. The cost to go $J^*(x)$ satisfies the principal of dynamic programming and even if the q term in the Bellman equation
>
> $$J^*(x) = \min_{u(\cdot)} \left\{ q(x, u(x)) + \gamma \mathop{\mathrm{E}}_{x' \sim \mathrm{P}(x'|x, u(x))} \left[J^*(f(x, u) + \epsilon) \right] \right\}.$$

(4) **(3 points)** Again, consider an infinite-horizon stochastic dynamic programming problem with finite states and finite controls, a run-time cost of $q(x, u)$ for state x and control u. Explain how policy evaluation can be used to compute the cost-to-go

of the policy

$$\pi = (u(\cdot), u(\cdot), u(\cdot), \dots).$$

For notation, you can use γ for the discount factor, $T_{x,x',u} = \mathrm{P}(x' \mid x, u)$ as the transition matrix of the MDP, and $J^\pi(x)$ as the cost-to-go from state x.

Answer: If we set up J as the vector of cost-to-go from each state x under policy π, and q is a vector of run-time costs $q(x, u(x))$ and T as the matrix $\mathrm{P}(x' \mid x, u(x))$, policy evaluation corresponds to the equation

$$J = q + \gamma T J$$
$$J = (I - \gamma T)^{-1} q.$$

END OF EXAM